



Telco Customer Churn Prediction

Project Documentation

Gorla Poojitha (GitHub: poojithareddy19)

4 October 2026

Contents

1. Executive summary

2. Business problem

- 2.1 Context
- 2.2 Objectives
- 2.3 Success criteria

3. Dataset

- 3.1 Source
- 3.2 Features
- 3.3 Cleaning
- 3.4 Data limitations

4. Segment and cohort analysis

- 4.1 Method
- 4.2 Findings

5. Modelling methodology

- 5.1 Preprocessing
- 5.2 Data split
- 5.3 Candidate models
- 5.4 Hyperparameter tuning
- 5.5 Calibration
- 5.6 Cost-based threshold
- 5.7 Profit model
- 5.8 Artifact

6. Results

- 6.1 Model comparison
- 6.2 Held-out performance
- 6.3 Uncertainty
- 6.4 Stability across random splits
- 6.5 Lift, capacity and calibration

7. Model interpretability

- 7.1 Permutation importance
- 7.2 Logistic regression odds ratios
- 7.3 Per-customer explanations with SHAP

8. Decision strategy and recommendations

- 8.1 Comparing the decision rules
- 8.2 Profit sensitivity to the offer success rate
- 8.3 Recommendations

9. Retention experiment design

9.1 Why an experiment is needed

9.2 Design

9.3 Sample size

9.4 Analysis plan

9.5 Phase 2: uplift modelling

10. Deployment

10.1 Streamlit application

10.2 Robustness

10.3 Hosting

11. Monitoring

12. Engineering

12.1 Code structure

12.2 Testing and continuous integration

12.3 Reproducibility

12.4 Design decisions

13. Limitations

14. Future work

Appendix A. How to run the project

Appendix B. Glossary

1. Executive summary

Repository	github.com/poojithareddy19/Customer-Churn-Prediction
Live demo	customer-churn-risk-app.streamlit.app
Licence	Apache-2.0
Stack	Python 3.11, scikit-learn, XGBoost, imbalanced-learn, SHAP, DuckDB, statsmodels, Streamlit, MLflow, pytest, GitHub Actions

Telecom operators lose a large share of revenue to customers who cancel their service (churn). A retention team can only act on customers it identifies in advance, and it has a limited budget, so it needs more than a yes/no label. It needs a trustworthy probability of churn for every customer, a rule that turns that probability into a contact decision, and an honest account of what the numbers can and cannot show.

This project delivers that for the IBM Telco Customer Churn dataset (7,043 customers, 26.5% churners):

- **A calibrated churn model.** Five candidate pipelines were compared with 5-fold stratified cross-validation and tuned with randomised search. A class-weighted XGBoost model won and was calibrated. On 1,409 held-out test customers it reaches **ROC-AUC 0.836 (95% CI 0.815 to 0.859)**, and across five random splits the test ROC-AUC averages 0.848.
- **Decision rules tied to business value.** A cost threshold (0.08), a profit threshold (0.34, the app default) and capacity rules (contact the top N% by risk) are all derived and compared. Contacting the riskiest 10% of customers reaches churners at 2.73 times the average rate.
- **Segment analysis in SQL.** Six DuckDB queries show where churn is concentrated: new customers, month-to-month contracts paid by electronic check, and fiber customers without security or support add-ons.
- **Explanations.** Permutation importance, logistic regression odds ratios and per-customer SHAP values explain both the model as a whole and individual scores.
- **An experiment design.** A pre-registered plan for a randomised test of a retention offer, with a power analysis (1,467 customers per arm for a 5 point lift), because a churn model alone cannot show that an offer works.
- **A deployed application.** A Streamlit app, live on Streamlit Community Cloud, returns a churn probability, a risk verdict, the expected cost of each decision and the top reasons behind the score.
- **Engineering practice.** Modular code in `src/`, 42 automated tests run on every push by GitHub Actions, drift monitoring with the population stability index, prediction logging, and MLflow experiment tracking.

The main caveat runs through the whole project: the data is a single observational snapshot. The model shows who is at risk, not what would keep them, and the cost and offer-success figures are assumptions until they are measured.

2. Business problem

2.1 Context

Acquiring a new telecom subscriber usually costs much more than keeping an existing one. A retention team can call customers, offer discounts or propose contract upgrades, but each contact costs money and staff time. Two kinds of mistake are possible:

- **False positive:** contacting a customer who would have stayed anyway. The cost is the offer and the effort, wasted.
- **False negative:** failing to contact a customer who then leaves. The cost is the lost revenue from that customer.

These two errors do not cost the same. Losing a customer typically costs far more than one unnecessary offer, so a decision threshold of 0.5 (the default in most classifiers) is the wrong choice.

2.2 Objectives

1. Rank customers by their probability of churning, with probabilities that are well calibrated (a predicted 30% should mean roughly 30% of such customers churn).
2. Turn the probabilities into a contact decision that reflects the relative cost of the two errors, and offer alternatives for a team with fixed capacity.
3. Explain why each customer receives their score, so the retention team can tailor the conversation.
4. Describe where churn is concentrated across the customer base.
5. Design the experiment that would measure whether a retention offer actually works.
6. Deploy the model behind a simple interface and make the whole pipeline reproducible and tested.

2.3 Success criteria

- The model must clearly beat a majority-class baseline. Predicting “nobody churns” scores 73.5% accuracy while finding zero churners, so accuracy is not used as the headline metric.
- Model selection uses ROC-AUC, which measures ranking quality independent of any threshold.
- Probability quality is measured with the Brier score and a calibration table, because the app shows probabilities to people.
- Every reported metric comes with an uncertainty estimate.

3. Dataset

3.1 Source

The IBM Telco Customer Churn sample dataset, as published on [Kaggle](#). The file `WA_Fn-UseC_-Telco-Customer-Churn.csv` is included in the repository.

- 7,043 customers, one row each
- 19 input features plus a customer ID and the target
- Target `Churn` (Yes/No): 1,869 churners (26.5%) and 5,174 non-churners (73.5%)

Money values in this document are in the units of the dataset's `MonthlyCharges` column, usually read as US dollars.

3.2 Features

Group	Features
Demographics	<code>gender</code> , <code>SeniorCitizen</code> , <code>Partner</code> , <code>Dependents</code>
Account	<code>tenure</code> (months), <code>Contract</code> , <code>PaperlessBilling</code> , <code>PaymentMethod</code> , <code>MonthlyCharges</code> , <code>TotalCharges</code>
Phone services	<code>PhoneService</code> , <code>MultipleLines</code>
Internet services	<code>InternetService</code> , <code>OnlineSecurity</code> , <code>OnlineBackup</code> , <code>DeviceProtection</code> , <code>TechSupport</code> , <code>StreamingTV</code> , <code>StreamingMovies</code>

Four columns are treated as numeric (`SeniorCitizen`, `tenure`, `MonthlyCharges`, `TotalCharges`) and fifteen as categorical.

3.3 Cleaning

Implemented in `src/features.py` :

- `customerID` is dropped, since it carries no predictive information.
- `Churn` is mapped to 1 (Yes) and 0 (No).
- `TotalCharges` is stored as text in the raw file and is converted to numeric. Eleven rows have a blank value. All eleven have zero tenure, meaning the customer had not yet been billed, so they are filled with 0.0 rather than dropped or imputed with a median.
- A median imputer remains inside the model pipeline as a safety net for unseen data.

3.4 Data limitations

- **Snapshot, no dates.** There is no time dimension, so the model can say whether a customer is likely to churn but not when, and drift over time cannot be observed.

- **Churn definition.** The dataset documentation describes `Churn` as customers who left within the last month, which is only a proxy for a longer retention window.
- **Observational.** No customer received an intervention, so no causal effect can be estimated from this data.

4. Segment and cohort analysis

4.1 Method

Six SQL queries in `sql/` run with DuckDB directly on the CSV file, covering all 7,043 customers. `scripts/segment_analysis.py` executes them, adds 95% Wilson score intervals to each churn rate (`src/segments.py`), and writes `reports/segment_analysis.md` plus one CSV per query in `reports/segments/`.

Query	Question
<code>01_overall.sql</code>	What is the overall churn rate?
<code>02_tenure_cohorts.sql</code>	How does churn change with customer tenure?
<code>03_contract_payment.sql</code>	Which contract and payment combinations churn most?
<code>04_service_mix.sql</code>	How do internet type and add-on services relate to churn?
<code>05_charge_bands.sql</code>	How does churn vary across monthly charge quintiles?
<code>06_revenue_at_risk.sql</code>	Where is the monthly revenue lost to churn concentrated?

4.2 Findings

Churn is concentrated early in the customer lifetime.

Tenure (months)	Customers	Churn rate	95% CI
0 to 6	1,481	52.9%	50.4% to 55.5%
7 to 12	705	35.9%	32.4% to 39.5%
13 to 24	1,024	28.7%	26.0% to 31.6%
25 to 48	1,594	20.4%	18.5% to 22.4%
49 and over	2,239	9.5%	8.4% to 10.8%

Contract type is the strongest single divider. Month-to-month customers churn at 42.7%, one-year customers at 11.3% and two-year customers at 2.8%, roughly a fifteen-fold difference.

One segment holds over half of all churners. Month-to-month customers paying by electronic check are 1,850 customers (26.3% of the base). They churn at 53.7% (95% CI 51.5% to 56.0%) and account for 53.2% of all churners.

Service mix matters. Fiber optic customers with neither online security nor tech support churn at 55.0%, while DSL customers with both churn at 6.3%. Customers who choose add-ons may differ in other ways, so this does not show that selling add-ons would reduce churn.

Churn rises with price, but not strictly. The cheapest fifth of customers (average 20.68 per month) churns at 9.2%. The fourth band (average 86.00) is highest at 36.1%, and the most expensive band (average 103.16) is slightly lower at 32.8%.

Revenue at risk is concentrated. Month-to-month contracts hold 86.9% of the monthly revenue lost to churn (120,847.10 of 139,130.85 per month).

Contract	Customers	Churn rate	Monthly revenue lost	Share of lost revenue
Month-to-month	3,875	42.7%	120,847.10	86.9%
One year	1,473	11.3%	14,118.45	10.1%
Two year	1,695	2.8%	4,165.30	3.0%

All of these findings are correlational. They show where churn is concentrated, not what would change it.

5. Modelling methodology

5.1 Preprocessing

A scikit-learn `ColumnTransformer` (`src/features.py`) scales the four numeric columns and one-hot encodes the fifteen categorical columns with `handle_unknown="ignore"` . The preprocessing is fitted inside each model pipeline, so:

- there are no separate encoder files to keep in sync with the model;
- no information from validation or test data leaks into training;
- the same preprocessing works for linear and tree models without implying a false order between categories.

5.2 Data split

A stratified 60/20/20 split with `random_state=42` (`src/train.py`):

Split	Customers	Used for
Training	4,225	Cross-validation, model comparison, tuning, fitting
Validation	1,409	Choosing the decision threshold only
Test	1,409	Final evaluation, used once at the end

Choosing the threshold on the test split would make the test metrics optimistic, which is why a separate validation split exists.

5.3 Candidate models

Five pipelines are compared by ROC-AUC under 5-fold stratified cross-validation on the training split. Two strategies handle the 73/27 class imbalance:

- **Class weighting:** `class_weight="balanced"` for logistic regression and Random Forest, and `scale_pos_weight` for XGBoost.
- **SMOTE oversampling:** applied through an `imblearn` pipeline, so synthetic examples are generated inside each training fold only and never leak into the fold being scored.

Candidate	Imbalance handling
Logistic regression	Class weights
Random Forest	Class weights
Random Forest	SMOTE
XGBoost	Class weights
XGBoost	SMOTE

5.4 Hyperparameter tuning

Each family is tuned with `RandomizedSearchCV` (10 iterations, 5-fold stratified CV, ROC-AUC scoring). The search spaces are:

Model	Parameters searched
Logistic regression	<code>C</code> on a log scale from 0.001 to 100 (20 values); penalty <code>l1</code> or <code>l2</code> ; <code>liblinear</code> solver
Random Forest	trees 200, 300, 500; max depth None, 8, 12, 16, 20; min samples split 2, 5, 10; min samples leaf 1, 2, 4; max features sqrt, log2, 0.5
Random Forest + SMOTE	as above, plus SMOTE neighbours 3 or 5
XGBoost	trees 150 to 500; depth 3 to 6; learning rate 0.01 to 0.1; subsample and column subsample 0.7 to 1.0; min child weight 1, 3, 5; L2 regularisation 1, 5, 10

The tuned pipeline with the highest mean cross-validated ROC-AUC is selected, with no preference for any model family.

5.5 Calibration

The winning pipeline is wrapped in `CalibratedClassifierCV` with sigmoid calibration. Tree ensembles and class weighting both distort raw scores, so calibration is what makes “30% probability” mean roughly 30% in practice. Calibration is fitted on the training split, the probabilities are scored on the validation split to choose the threshold, and the final model is recalibrated on training plus validation data.

5.6 Cost-based threshold

Each customer is contacted if their probability is at or above a threshold. The cost of a threshold on the validation split is:

expected cost = (false positives x cost of a wasted offer) + (false negatives x cost of a missed churner)

The defaults, set in `src/features.py` and overridable with environment variables, are derived from the dataset's mean monthly charge of about 65:

Assumption	Default	Environment variable
Wasted offer (one month of revenue)	65	<code>CHURN_FALSE_POSITIVE_COST</code>
Missed churning (twelve months of revenue)	780	<code>CHURN_FALSE_NEGATIVE_COST</code>

With a 12:1 cost ratio the theoretical break-even probability is $65 / (65 + 780)$, about 7.7%. Scanning thresholds on the validation split picks **0.08**, with an expected cost of 45,760 (560 false positives, 12 false negatives). This threshold is saved with the model.

5.7 Profit model

The cost rule assumes that contacting a churning always prevents the loss, which is unrealistic. The profit model relaxes this with an explicit offer success rate:

profit = sum over contacted churning of (success rate x monthly charge x retention months) minus (offer cost x customers contacted)

Assumption	Default	Environment variable
Offer cost	65	<code>CHURN_OFFER_COST</code>
Offer success rate	30%	<code>CHURN_OFFER_SUCCESS_RATE</code>
Months of revenue kept per saved customer	12	<code>CHURN_RETENTION_MONTHS</code>

The threshold that maximises validation profit at a 30% success rate is **0.34**. Because the success rate is a pure assumption, the training report also includes a sensitivity analysis at 10%, 20%, 30% and 50% (section 8.2).

5.8 Artifact

The calibrated pipeline, the cost threshold, the feature column order and the full evaluation report are saved together to `artifacts/churn_model.joblib`. A readable copy of the report is written to `artifacts/training_report.json`, stamped with the run time and git commit. If MLflow is installed, each training run is also logged to a local MLflow store.

6. Results

All figures come from `artifacts/training_report.json` and the files in `reports/`. Threshold-dependent metrics are reported at the 0.08 cost threshold saved with the model unless stated otherwise.

6.1 Model comparison

Cross-validated ROC-AUC on the training split, mean and standard deviation over five folds:

Pipeline	Defaults	Tuned
Logistic regression (balanced)	0.8443 ± 0.0214	0.8444 ± 0.0212
Random Forest (class-weighted)	0.8227 ± 0.0132	0.8453 ± 0.0141
Random Forest + SMOTE	0.8216 ± 0.0090	0.8437 ± 0.0139
XGBoost + SMOTE	0.8157 ± 0.0132	0.8453 ± 0.0143
XGBoost (class-weighted)	0.8132 ± 0.0102	0.8481 ± 0.0159

Class-weighted XGBoost was selected after tuning: 350 shallow trees of depth 4, learning rate 0.01, subsample 0.85, column subsample 0.7, minimum child weight 1, L2 regularisation 5.

Two observations matter for interpretation. First, tuning lifts the tree models substantially but barely moves logistic regression (best `C` = 2.64, `l2` penalty). Second, the tuned models sit within 0.004 ROC-AUC of each other, while the fold-to-fold standard deviations are around 0.014 to 0.021. The ranking among the top candidates is therefore not decisive, and a simpler logistic regression would perform almost as well.

6.2 Held-out performance

Metric	Validation (1,409)	Test (1,409)
ROC-AUC	0.8596	0.8361
Brier score	0.1302	0.1402
Cost-optimal threshold	0.08	0.05
Recall (churn) at 0.08	0.968	0.944
Precision (churn) at 0.08	0.39	0.40
Accuracy at 0.08	0.594	0.606

Test confusion matrix at the 0.08 threshold:

	Predicted: stays	Predicted: churns
Actually stays	501	534
Actually churns	21	353

The low accuracy is intended. At a 12:1 cost ratio, the cost rule accepts many unnecessary contacts to miss as few churners as possible: it catches 353 of 374 churners. The test split's own cost curve would prefer 0.05, which shows the cost curve is flat in this region and the exact threshold value should not be over-interpreted.

6.3 Uncertainty

Bootstrap 95% percentile intervals on the test split (1,000 resamples, seed 42):

Metric	Estimate	95% CI
ROC-AUC	0.8361	0.8146 to 0.8590
Brier score	0.1402	0.1295 to 0.1512
Recall	0.9439	0.9208 to 0.9648
Precision	0.3980	0.3641 to 0.4304
Expected cost	51,090	44,525 to 57,658
Top-decile lift	2.73	2.45 to 3.04

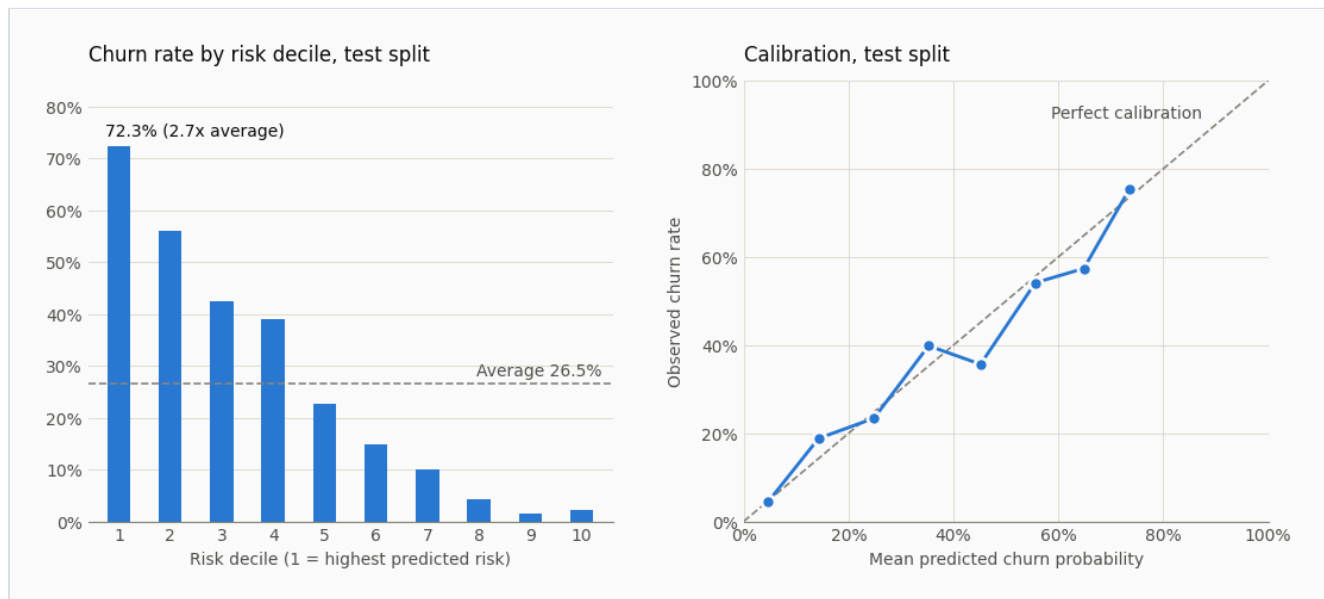
6.4 Stability across random splits

`scripts/seed_stability.py` repeats the split, calibration and evaluation with five seeds:

Seed	Test ROC-AUC	Test Brier	Test recall
0	0.8515	0.1329	0.9706
1	0.8610	0.1289	0.9786
2	0.8482	0.1349	0.8984
3	0.8434	0.1362	0.9866
42	0.8361	0.1402	0.9439
Mean ± std	0.8480 ± 0.0093	0.1346 ± 0.0042	0.9556 ± 0.0358

The deployed split (seed 42) is at the low end, so the headline 0.836 is not a lucky result. Recall varies more than ROC-AUC because the threshold chosen on each validation split varies between 0.04 and 0.12.

6.5 Lift, capacity and calibration



Left: test churn rate by risk decile against the 26.5% average. Right: predicted probability against observed churn rate, close to the diagonal.

Ranking test customers by predicted probability and splitting them into ten equal groups:

Decile	Churn rate	Lift	Cumulative share of churners
1 (highest risk)	72.3%	2.73	27.3%
2	56.0%	2.11	48.4%
3	42.6%	1.60	64.4%
4	39.0%	1.47	79.1%
5	22.7%	0.85	87.7%
6 to 10	1.4% to 14.9%	0.05 to 0.56	93.3% to 100%

For a team that can only make a fixed number of contacts:

Capacity	Customers contacted	Probability cutoff	Recall	Precision	Expected cost
5%	71	0.720	0.155	0.817	247,325
10%	141	0.669	0.273	0.723	214,695
20%	282	0.561	0.484	0.642	157,105
30%	423	0.413	0.644	0.570	115,570
40%	564	0.294	0.791	0.525	78,260
50%	705	0.160	0.877	0.465	60,385

The calibration chart shows predicted probabilities tracking observed churn rates closely across the range, which supports showing raw probabilities to users.

7. Model interpretability

7.1 Permutation importance

The drop in test ROC-AUC when each column is randomly shuffled (mean of 10 repeats):

Feature	Importance
Contract	0.087
tenure	0.030
InternetService	0.016
MonthlyCharges	0.010
TotalCharges	0.007
OnlineSecurity	0.005
TechSupport	0.002
StreamingMovies	0.002

Permutation importance is used instead of the trees' built-in (Gini) importance because it is measured on held-out data and is not biased toward continuous or high-cardinality columns. Contract type is by far the strongest signal, consistent with the segment analysis.

7.2 Logistic regression odds ratios

The tuned logistic regression gives an interpretable cross-check. Numeric features are standardised, so their odds ratios are per one standard deviation:

Feature	Odds ratio	Reading
tenure (per SD)	0.29	Longer tenure, much lower odds of churn
Contract = Two year	0.45	Lower odds than other contracts
MonthlyCharges (per SD)	0.46	See the note on correlated features below
InternetService = Fiber optic	2.02	About twice the odds
Contract = Month-to-month	1.93	About twice the odds
TotalCharges (per SD)	1.87	See the note on correlated features below
InternetService = DSL	0.54	Lower odds

`tenure`, `MonthlyCharges` and `TotalCharges` are strongly correlated (total charges are roughly tenure times monthly charges), so their individual coefficients should not be read in isolation.

7.3 Per-customer explanations with SHAP

The app's "Why this score?" panel runs `shap.TreeExplainer` on the XGBoost model inside the first calibration fold. One-hot columns are summed back to their original feature, so the user sees "Contract = Month-to-month" rather than an encoded column name. The top five contributions are shown in log-odds with a "raises risk" or "lowers risk" label. The values describe the direction and relative size of each feature's effect on the uncalibrated score, not exact changes in the displayed probability.

8. Decision strategy and recommendations

8.1 Comparing the decision rules

On the test split:

Rule	Customers contacted	Churners reached	Precision
Top 10% by risk	10%	27.3%	72.3%
Top 20% by risk	20%	48.4%	64.2%
Top 40% by risk	40%	79.1%	52.5%
Profit threshold 0.34 (app default)	37.1%	75.1%	53.7%
Cost threshold 0.08	63.0%	94.4%	39.8%

8.2 Profit sensitivity to the offer success rate

Assumed success rate	Threshold	Customers contacted	Test profit
10%	0.67	139	508.84
20%	0.46	379	17,064.64
30% (default)	0.34	523	42,869.50
50%	0.18	674	102,068.30

At the 0.08 cost threshold the 30% scenario earns 38,107.70 on the test split, less than the 42,869.50 from the profit threshold. At a 10% success rate the campaign barely breaks even.

8.3 Recommendations

1. **Use the profit threshold (0.34) as the starting rule.** The cost threshold is cheapest only under its own assumption that every contacted churner is saved. The profit threshold contacts far fewer customers and earns more under a realistic success rate. The app uses it by default.
2. **With a fixed team size, contact the top N% by risk.** The top 20% reaches almost half of all churners at 2.4 times the average churn rate.
3. **Focus early-life and month-to-month customers.** Customers in their first six months and month-to-month customers paying by electronic check carry most of the churn and most of the lost revenue.
4. **Measure the offer success rate before scaling.** It is the input that most changes the answer, swinging profit from break-even to over 100,000 on 1,409 customers. Section 9 describes how to measure it.

9. Retention experiment design

The full plan is in `docs/experiment_plan.md`; this section summarises it. The sizing inputs are calculated by `scripts/experiment_sizing.py` and written to `reports/experiment_sizing.md`.

9.1 Why an experiment is needed

The churn model predicts who is likely to leave. It does not predict who will stay because of an offer. Customers fall into four groups:

Group	Without offer	With offer	Value of an offer
Persuadable	leaves	stays	Positive: the offer causes retention
Sure thing	stays	stays	Wasted cost
Lost cause	leaves	leaves	Wasted cost
Sleeping dog	stays	leaves	Negative: the contact prompts them to leave

A high churn score mixes persuadables with lost causes. Only a randomised test separates the effect of the offer from who was chosen to receive it.

9.2 Design

Element	Decision
Hypothesis	Among flagged customers, a retention offer changes the 90-day retention rate (two-sided, so harm is also detected)
Unit	The customer account, assigned by a salted hash of the customer ID
Eligibility	Customers at or above the model threshold, with exclusions fixed before launch (existing offers, open complaints or collections, staff and test accounts)
Arms	50% control (no proactive offer), 50% treatment (the offer)
Primary metric	90-day retention rate, intention to treat
Secondary metrics	Revenue retained per assigned customer; contract upgrades
Guardrails	Offer cost per incremental retained customer; complaint and downgrade rates

9.3 Sample size

On the test split, 887 of 1,409 customers (63.0%) are eligible, and their observed retention rate is 60.2%. With a two-sided test at alpha 0.05 and power 0.8 (`sample_size_two_proportions` in `src/experiment.py`):

Minimum detectable effect	Customers per arm	Total
+2 percentage points	9,318	18,636
+3 percentage points	4,121	8,242
+5 percentage points	1,467	2,934
+10 percentage points	355	710

The planned minimum detectable effect is +5 points. Applying the 63.0% eligible share to all 7,043 customers gives about 4,434 eligible customers, enough to enrol 2,934 at once. With the 90-day window plus about two weeks for data to settle, the test runs for roughly 15 weeks.

9.4 Analysis plan

Fixed before launch:

1. Run a sample ratio mismatch check (`srm_check` , chi-square). A p-value below 0.001 means assignment or logging is broken, and the results are not used.
2. Compare 90-day retention with a two-proportion z-test (`analyze_ab`) and report the absolute difference with its 95% confidence interval.
3. Roll out only if the lower bound of the interval is above zero and the guardrails pass.
4. Report segment cuts as exploratory only.

The plan also covers peeking (the analysis runs once, at the end), novelty effects, contamination between arms, regression to the mean, and freezing the model during the test.

9.5 Phase 2: uplift modelling

Once the experiment has run, its randomised data can train a model of the offer effect itself, using either a two-model approach (separate models for treated and control customers) or a class-transformation approach (a single classifier on a transformed target). Uplift models are evaluated with Qini curves, and the offer then goes to the customers most likely to be persuaded, not simply the riskiest.

10. Deployment

10.1 Streamlit application

`app.py` is a Streamlit app that loads the saved artifact and is live at customer-churn-risk-app.streamlit.app.

Telco Customer Churn Prediction

A raw-feature pipeline trained from the Telco churn dataset and selected by ROC-AUC.

Selection metric: ROC-AUC. Validation thresholds: profit 0.34, expected cost 0.08. Cost assumptions: wasted offer = 65, missed churner = 780.

Model summary

Lift and capacity tables (test split)

Decision rule

☒ Profit threshold ☐ Cost threshold ☐ Capacity (top N%)

Threshold that maximised validation profit assuming 30% of contacted churners are saved. The success rate is an assumption, not measured data.

Customer Details

Gender	Senior Citizen	Partner
Female	0	No
Dependents	Phone Service	Multiple Lines
No	Yes	No
Internet Service	Online Security	Online Backup
Fiber optic	No	No
Device Protection	Tech Support	Streaming TV
No	No	No
Streaming Movies	Contract	Paperless Billing
No	Month-to-month	Yes
Payment Method	Tenure (months)	Monthly Charges
Electronic check	12	70.00
Total Charges		
1500.00		

Decision threshold

0.00 0.34 1.00

Predict churn risk

Prediction Result

High churn risk: 56.5% probability

Expected cost if not contacted	Expected cost if contacted
440	28

Decision rule: Profit threshold

Prediction threshold used: 0.340

Why this score?

feature	contribution	direction
Contract = Month-to-month	0.5621	raises risk
InternetService = Fiber optic	0.2869	raises risk
PaymentMethod = Electronic check	0.2102	raises risk
OnlineSecurity = No	0.1409	raises risk
TotalCharges = 1500.0	-0.1266	lowers risk

SHAP values for the uncalibrated model score, in log-odds. They show the direction and relative size of each feature's push on the score, not exact changes in the displayed probability. One-hot columns are summed back to the original feature.

Applied input guard

The app scoring a month-to-month fiber customer: 56.5% churn probability, flagged as high risk at the 0.34 profit threshold, with the top SHAP reasons.

Features:

- **All 19 inputs** in a three-column form, built from a single form schema in `src/features.py`.

- **Consistency guards.** No internet service forces every internet add-on to “No internet service”, and no phone service forces multiple lines to “No phone service”, so the model never sees impossible combinations.
- **Three decision rules.** The profit threshold (default, 0.34) and the cost threshold (0.08), both adjustable with a slider, and a capacity rule that applies the cutoff for the chosen top N%.
- **Result panel.** Churn probability, a high or lower risk verdict, the expected cost of contacting and of not contacting the customer, and the rule and threshold used.
- **“Why this score?”** The top five SHAP contributions for the customer.
- **Model summary** and the lift and capacity tables, available in expandable sections.
- **Prediction logging.** Every prediction is appended to `logs/predictions.jsonl` with a UTC timestamp, the inputs, the probability, the rule, the threshold and the verdict. A logging failure never blocks a prediction.

10.2 Robustness

- The app builds its input frame from the feature order saved with the model, so the form and the model cannot drift apart.
- If the artifact is missing, the app trains a model on first load and shows a warning.
- The artifact is committed (2.4 MB), so the cloud deployment starts without retraining.

10.3 Hosting

The app runs on Streamlit Community Cloud from the `main` branch with Python 3.11, and redeploys automatically on every push.

11. Monitoring

`scripts/drift_check.py` computes the population stability index (PSI, `src/monitoring.py`) for `tenure`, `MonthlyCharges`, `TotalCharges` and the predicted probability. PSI compares the distribution of a column in a reference sample with a current sample, using 10 quantile bins of the reference.

PSI	Reading
Below 0.1	Stable
0.1 to 0.25	Moderate shift
Above 0.25	Significant shift, review or retrain

Because the dataset has no time dimension, real drift cannot be measured. The report demonstrates the method on two comparisons:

Column	Training vs test	Training vs simulated shift (month-to-month only)
tenure	0.0068 (stable)	0.5781 (significant)
MonthlyCharges	0.0078 (stable)	0.1050 (moderate)
TotalCharges	0.0106 (stable)	0.2842 (significant)
Predicted probability	0.0068 (stable)	3.6921 (significant)

The simulated shift keeps only month-to-month customers, to mimic a customer base moving toward short contracts. It is labelled as simulated in the report. In production, the reference would be the training data and the current sample would be each new month of scored customers, read from the prediction log.

12. Engineering

12.1 Code structure

Module	Responsibility
<code>src/features.py</code>	Data loading and cleaning, preprocessing, form schema, input guards, cost and profit assumptions
<code>src/train.py</code>	Splitting, candidate comparison, tuning, calibration, threshold selection, profit analysis, artifact and report export, MLflow logging
<code>src/evaluate.py</code>	Metrics, cost curve, calibration table, permutation importance, lift and capacity tables, bootstrap intervals, odds ratios, profit curve
<code>src/app_helpers.py</code>	SHAP explanations, prediction logging, reading costs and the profit threshold from the report
<code>src/segments.py</code>	DuckDB query runner and Wilson intervals
<code>src/experiment.py</code>	Sample size, sample ratio mismatch check, two-proportion test
<code>src/monitoring.py</code>	Population stability index
<code>app.py</code>	Streamlit interface

Scripts in `scripts/` regenerate each report:

Script	Output
<code>seed_stability.py</code>	<code>reports/seed_stability.csv</code>
<code>segment_analysis.py</code>	<code>reports/segment_analysis.md</code> , <code>reports/segments/*.csv</code>
<code>experiment_sizing.py</code>	<code>reports/experiment_sizing.md</code>
<code>drift_check.py</code>	<code>reports/drift_report.md</code>
<code>readme_charts.py</code>	<code>docs/images/model_charts_light.png</code> and <code>_dark.png</code>

12.2 Testing and continuous integration

42 pytest tests across 7 modules cover feature cleaning and guards, the training pipeline and artifact, evaluation functions, app helpers (including SHAP output and prediction logging), the SQL queries and Wilson intervals, the experiment statistics and the PSI calculation. GitHub Actions runs the full suite on every push and pull request, on Ubuntu 24.04 with Python 3.11.

12.3 Reproducibility

- Every random step uses a fixed seed (42 by default).
- Dependency versions are pinned in `requirements.txt`; `requirements-dev.txt` adds pytest and MLflow.
- The training report records the run timestamp, git commit and whether the working tree had uncommitted changes.
- All reports are generated by scripts, and the README lists the commands to regenerate them.

12.4 Design decisions

Decision	Rationale
One-hot encoding inside the pipeline	No separate encoder files, no false ordering, reusable by linear and tree models
SMOTE inside the cross-validation pipeline	Resampling happens per fold on training data only, so CV scores stay honest
ROC-AUC as the selection metric	Threshold-independent, so model choice is separate from the business cost trade-off
Separate validation split for the threshold	The threshold is a hyperparameter; choosing it on test data would bias test metrics
Calibrate before choosing the threshold	The threshold must be picked on the same probability scale the app shows
Cost and profit thresholds	Make the trade-off explicit and tunable instead of hiding it behind 0.5
Capacity rule	Retention teams usually have a fixed number of calls to make
Bootstrap intervals and seed stability	One test split of 1,409 customers gives a noisy estimate
Permutation importance	Measured on held-out data and not biased toward continuous columns
<code>TotalCharges</code> blanks filled with 0.0	All 11 rows have zero tenure and had not yet been billed
Feature order saved with the model	The app's input frame always matches what the model expects

13. Limitations

- **Correlation, not causation.** Neither the model nor the segment analysis shows that any action would reduce churn. A high churn score does not mean a customer will respond to an offer.
- **Assumed costs.** The offer cost, the cost of a missed churning and the offer success rate are estimates derived from the mean monthly charge, not measured values. Both thresholds depend on them.
- **Static snapshot.** With no dates, the model cannot say when a customer will churn, and drift can only be demonstrated with a simulated shift.
- **Churn definition.** The target covers customers who left within the last month, a proxy for the 90-day retention the experiment would measure.
- **Small randomised search.** Ten iterations per model family. The tuned models are within 0.005 ROC-AUC of each other, so a larger search could change the winner.
- **Public sample data.** The IBM dataset is a sample for teaching, so results may not carry over to a real operator's customer base.

14. Future work

- Run the retention experiment, measure the real offer success rate, and feed it into the profit threshold.
- Fit an uplift model on the experiment data and compare uplift targeting with risk targeting.
- Replace SMOTE with SMOTENC, which handles categorical features directly.
- Move the exploratory notebook onto the `src/` helpers so it runs locally.
- With timestamped data, model time to churn (survival analysis) and monitor real drift monthly from the prediction log.

Appendix A. How to run the project

```
git clone https://github.com/poojithareddy19/Customer-Churn-Prediction.git
cd Customer-Churn-Prediction
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

On macOS or Linux, activate with `source .venv/bin/activate`.

Task	Command
Run the app	<code>python -m streamlit run app.py</code>
Retrain the model	<code>python -m src.train</code>
Regenerate the reports	<code>python scripts/seed_stability.py</code> , then <code>segment_analysis.py</code> , <code>experiment_sizing.py</code> , <code>drift_check.py</code> and <code>readme_charts.py</code>
Run the tests	<code>pip install -r requirements-dev.txt</code> , then <code>python -m pytest -q</code>
View MLflow runs	<code>mlflow ui --backend-store-uri</code> <code>sqlite:///mlflow.db</code>

Appendix B. Glossary

Term	Meaning
Churn	A customer cancelling their service
ROC-AUC	The probability that a randomly chosen churner receives a higher score than a randomly chosen non-churner; 0.5 is random, 1.0 is perfect
Brier score	Mean squared difference between predicted probabilities and actual outcomes; lower is better
Calibration	How closely predicted probabilities match observed rates
Recall	Share of actual churners the rule flags
Precision	Share of flagged customers who actually churn
Lift	Churn rate in a group divided by the overall churn rate
SHAP	A method that splits a model's score into contributions from each feature
PSI	Population stability index, a measure of how much a distribution has shifted
SRM	Sample ratio mismatch, when arm sizes differ from the planned split more than chance allows
MDE	Minimum detectable effect, the smallest true effect an experiment is powered to detect
Uplift	The change in a customer's outcome caused by a treatment, such as an offer